

Introduction to Machine Learning

with Examples in Power Systems

Dr. Wangkun Xu
`wangkun.xu18@imperial.ac.uk`

Department of EEE, Imperial College London

July. 22, 2026

Outline I

Power-System and Machine Learning Background

- Course Overview

- Introduction to Electric Power System

- Machine Learning: Some Examples

- Taxonomy of Machine Learning

- Copyright and Reference

Supervised Learning

- Overview

- Linear Regression

- Logistic Regression

- Neural Networks

- Support Vector Machine

Unsupervised Learning

- Overview

- Principal Component Analysis

- k-means Clustering

Outline II

Reinforcement Learning

- Overview Only

Performance Measure

- Performance Measure

- Underfit and Overfit

- Regularization

- Hyperparameters

- Validation Set

- Summary

Special Topic One: Time Series Foundation Model

- No Free Lunch Theorem

- What is Foundation Model?

- Time Series Foundation Model

- Amazon Chronos

Special Topic Two: DATA? DATA!

- Why Data is Important?

Outline III

From Raw Data to an ML Dataset
European's Effort on Data Transparency

Appendix

Linear Regression: Maximum Likelihood Estimation
Soft-margin SVM and Kernel SVM

Outline

Power-System and Machine Learning Background

Course Overview

Introduction to Electric Power System

Machine Learning: Some Examples

Taxonomy of Machine Learning

Copyright and Reference

Overview I

- ▶ 4-5 hour lectures to cover the *BASIC* idea of machine learning, deep learning, and a little bit on *foundation model*, with application to energy sector, especially power systems.
- ▶ Some coding tutorial. Please go to <https://github.com/xuwkk/intro2ml-coding-tutorial> to get the code!
- ▶ About myself: Dr. *Wangkun Xu*
 - ▶ Research Associate in the Department of EEE.
 - ▶ Obtained my PhD and M.Sc. in 2024 and 2019 at Imperial.
 - ▶ Main research focus is machine learning and optimization for robust and secure power system operations with renewables.
- ▶ Contact me via email regarding on the course or any inquiry. Please share with me any typo in the slides and suggestions of this course!

Power-System and Machine Learning Background

Learning objectives

By the end of this section, you should be able to:

- ▶ identify the main components of an electric power system;
- ▶ explain why electricity supply and demand must remain balanced;
- ▶ distinguish the concepts on frequency, voltage, and power flow;
- ▶ understand why renewable energy creates operational uncertainty;
- ▶ identify where data and artificial intelligence can support the grid.

Outline

Power-System and Machine Learning Background

Course Overview

Introduction to Electric Power System

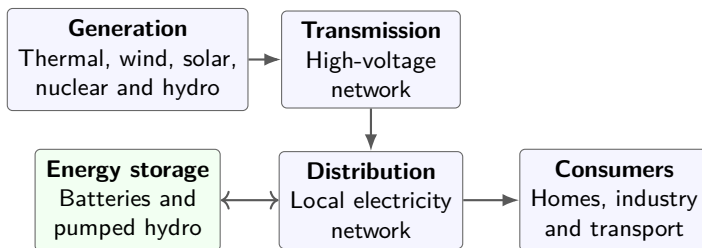
Machine Learning: Some Examples

Taxonomy of Machine Learning

Copyright and Reference

What Is an Electric Power System?

An electric power system delivers electrical energy from energy sources to consumers through an interconnected network.

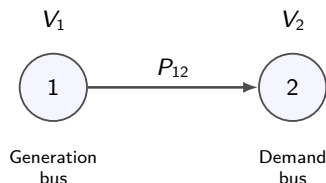


Frequency, Voltage, and Power Flow I

Frequency A system-wide indicator of the balance between active-power supply and demand.

Voltage The electrical potential at a particular location. Voltage must remain within an acceptable operating range.

Power flow The movement of electrical power through transmission lines and transformers.



Frequency, Voltage, and Power Flow II

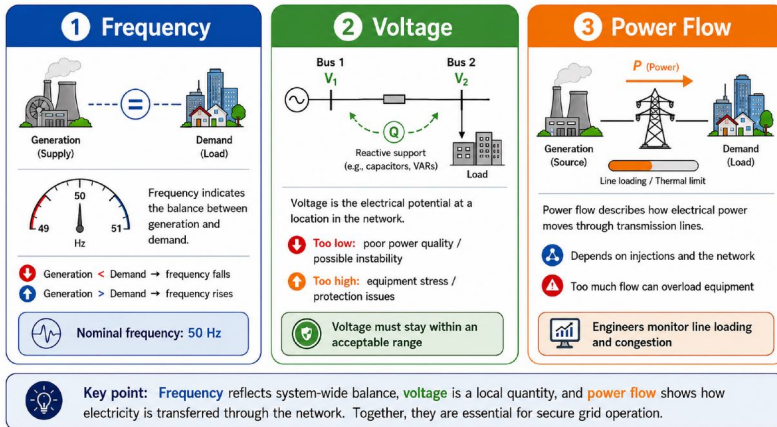


Figure: Power system frequency, voltage, and power flow.

Electricity Supply and Demand Must Balance

At every moment, the electrical power supplied to the grid must approximately match the power consumed:

$$\sum_i P_i^{\text{gen}} + \sum_j P_j^{\text{ren}} = P^{\text{demand}} + P^{\text{loss}}. \quad (1)$$

Too little generation

$$P^{\text{supply}} < P^{\text{demand}}$$

System frequency tends to decrease.

Too much generation

$$P^{\text{supply}} > P^{\text{demand}}$$

System frequency tends to increase.

Engineering interpretation

System frequency provides an important indication of the balance between generation and demand. Its nominal value in Great Britain is 50 Hz. The accepted operational range is 49.8 Hz to 50.2 Hz.

What Decisions Do System Operators Make?

Power-system operation involves decisions over several time scales.

Time scale	Decision	Main question
Days to hours	Generator scheduling	Which generating units should be available?
Hours to minutes	Economic dispatch	How much power should each generator produce?
Minutes to seconds	Balancing and reserves	How should unexpected demand or renewable changes be managed?
Seconds or faster	Control and protection	How should the system respond to a fault or instability?

A typical engineering objective

Operate the system safely and reliably while minimising generation cost, emissions, and renewable-energy curtailment.

Power Systems are Becoming More and More Renewable and Digital

You are probably aware that the new trend of large penetration of **renewable energy** and **digitalization** of the grid.



Why Does Renewable Energy Make Operation More Difficult? I

Variability

Renewable generation changes with:

- ▶ wind speed;
- ▶ solar irradiance;
- ▶ cloud cover;
- ▶ time of day;
- ▶ season and weather.

Forecast uncertainty

Future renewable output is not known exactly:

$$P_{t+1}^{\text{ren}} = \hat{P}_{t+1}^{\text{ren}} + \varepsilon_{t+1},$$

where:

- ▶ $\hat{P}_{t+1}^{\text{ren}}$ is the forecast;
- ▶ ε_{t+1} is the forecast error.

Possible consequences

Forecast errors can lead to insufficient reserves, higher costs, renewable curtailment, transmission congestion, or security problems.

Why Does Renewable Energy Make Operation More Difficult? II

The renewable assets can cause stability issues in the grid.

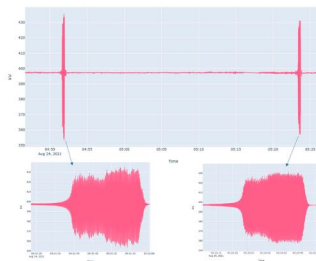


Figure: Sub-synchronous voltage oscillations (8Hz) recorded in GB transmission system, North Scotland, in 2021 [Source: NESO Report on Sub-synchronous oscillations in GB].

Power Systems Are Becoming More Digital

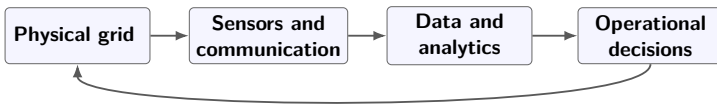
Modern power systems produce large quantities of data. But how to *get useful information* from the data is one thing we need to consider.

Grid measurements

- ▶ voltage and current;
- ▶ active and reactive power;
- ▶ system frequency;
- ▶ generator output;
- ▶ line and transformer status.

External and operational data

- ▶ weather forecasts;
- ▶ electricity demand;
- ▶ energy prices;
- ▶ equipment records;
- ▶ operator actions and alarms.



Outline

Power-System and Machine Learning Background

Course Overview

Introduction to Electric Power System

Machine Learning: Some Examples

Taxonomy of Machine Learning

Copyright and Reference

From Power-System Problems to Machine Learning

Based on your understanding of the power system concepts, consider the following engineering questions:

1. How much wind or solar power will be available tomorrow?
2. Will a transmission line become overloaded?
3. Is the current operating condition stable or unstable?
4. Are any sensor measurements unusual or corrupted?
5. Which control action should be taken after a disturbance?

Discussion

For each question, identify:

- ▶ the possible input data;
- ▶ the required output;
- ▶ whether the output is numerical, categorical, or a control action.

Transition

We now introduce machine learning and classify these engineering problems into different learning tasks.

Machine Learning: A Less Formal Definition I

Answering these questions requires methods that can learn useful patterns from data and use them for prediction, classification, or decision support. This is one of the main roles of *machine learning* (ML).

Definition

(Less formal). Machine learning uses data to learn a mapping from inputs to useful outputs, without explicitly programming every rule.

The essentials include:

- ▶ **Data:** examples from which the model learns;
- ▶ **Model:** a function that converts inputs into outputs.
- ▶ **Training:** a procedure to get the model converting inputs into outputs;
- ▶ **Evaluation:** checking whether it works on unseen data.

And we will introduce ML based on the four factors.

Machine Learning: Some Examples I

Machine learning is a very broad and practical topic. Therefore, this lecture only covers a selection of fundamental and practical concepts (with a little *time series foundation model*) part. The underlying ideas also provide a foundation for more advanced methods such as deep learning.

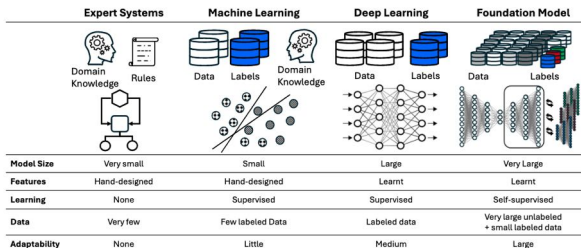


Figure: The evolution of AI (Source: [Hamann et al., 2024]).

Machine Learning: Some Examples II

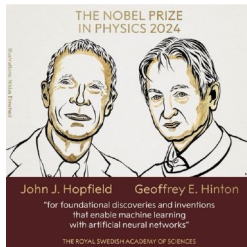
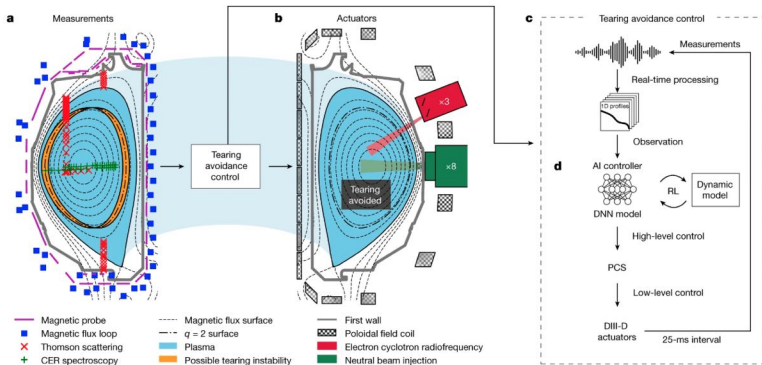


Figure: The Royal Swedish Academy of Sciences has decided to award the Nobel Prize in Physics 2024 to John J. Hopfield and Geoffrey Hinton *for foundational discoveries and inventions that enable machine learning with artificial neural networks.*

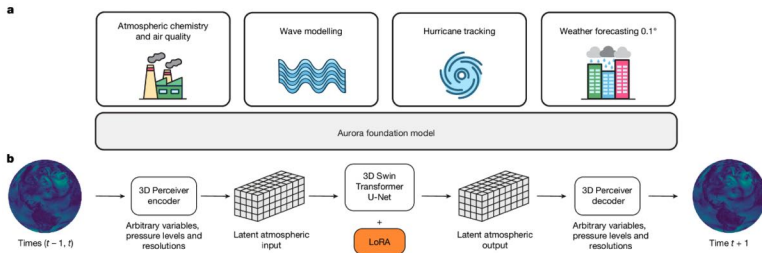
Machine Learning: Some Examples III



a. The selected diagnostic systems used in this work: magnetics, Thomson scattering (TS) and charge-exchange recombination (CER) spectroscopy. The possible tearing instability of $m/n = 2/1$ is shown in orange. **b.** The heating, current drive and control actuators used in this work. **c.** Schematic description of the tearing-avoidance control, including preprocessing, high-level control by a DNN and low-level control by a PCS. **d.** The AI controller based on the DNN.

Figure: Avoiding fusion plasma tearing instability with deep reinforcement learning. Source: Nature.

Machine Learning: Some Examples IV



Icons are for illustrative purposes only. **a**, Aurora is pretrained on several heterogeneous datasets with different resolutions, variables and pressure levels. The model is then fine-tuned for several operational forecasting scenarios at different resolutions: atmospheric chemistry and air quality at 0.4° , wave modelling at 0.25° , hurricane tracking at 0.25° and weather forecasting at 0.1° . **b**, Aurora is a flexible 3D Swin Transformer¹⁹ with 3D Perceiver-based²¹ atmospheric encoders and decoders. The model is able to ingest inputs with different spatial resolutions, numbers of pressure levels and variables.

Figure: A foundation model for the Earth system. Source: Nature.

Outline

Power-System and Machine Learning Background

Course Overview

Introduction to Electric Power System

Machine Learning: Some Examples

Taxonomy of Machine Learning

Copyright and Reference

Machine Learning Taxonomy I

A formal definition

A computer program is said to learn from experience E with respect to some class of task T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

This definition identifies three questions:

1. **Task T :** What output do we want the model to produce?
2. **Experience E :** What information is available during learning?
3. **Performance P :** How do we measure whether the model works well?

Based on the definition, machine learning algorithms can be classified into,

- ▶ Based on the *learning task T* , we roughly have
 - ▶ *Regression Task*
 - ▶ *Classification Task*

Machine Learning Taxonomy II

- ▶ Based on the kind of *experience* E the data can have during training, we have
 - ▶ Supervised Learning
 - ▶ Unsupervised Learning
 - ▶ Reinforcement Learning (will not be covered).
- ▶ In order to *evaluate the performance* P of a machine learning algorithm, we must design various quantitative measures of its performance.

Learning Tasks: Regression and Classification I

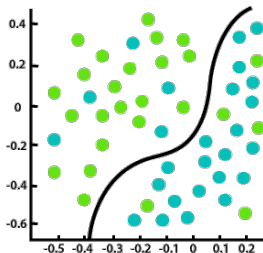
Regression

The computer program is asked to predict a numerical/float value given some input. To solve this task, the learning algorithm is asked to output a function f : $\mathbb{R}^D \rightarrow \mathbb{R}^I$.

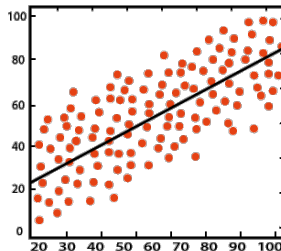
Classification

The computer program is asked to specify which of k categories some input belongs to. The learning algorithm is usually asked to produce a function $f : \mathbb{R}^D \rightarrow \{1, \dots, k\}$. Sometimes, f can produce a probability distribution over classes $p(y = c \mid \mathbf{x})$.

Learning Tasks: Regression and Classification II



Classification



Regression

Figure: Regression vs classification¹.

Learning Tasks: Regression and Classification III

Question

Determine the tasks T of the following activities in energy system (regression or classification?):

1. Renewable generation or electricity consumption forecast.
2. Detect if there is a congestion in the electricity transmission line.
3. Predict the optimal dispatch plan of generator.
4. Determine if the power system operational point is stable or not.

¹<https://www.javatpoint.com/regression-vs-classification-in-machine-learning> ▶

Learning Experience: How Is the Model Trained? I

Supervised learning

Training data contain input–output pairs:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N.$$

The model learns to predict y from $\mathbf{x}$.

Unsupervised learning

Training data contain only inputs:

$$\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N.$$

The goal is to discover useful structure or patterns.

Reinforcement learning

An agent interacts with an environment and learns from rewards.

Overview only in this lecture.

Learning Experience: How Is the Model Trained? II

Question

Determine the experience E of the following activities in energy system (supervised or unsupervised?):

1. Train renewable generation forecaster based on historical data.
2. Compare the common patterns of UK's wind generation in summer and wind.

Where Can Artificial Intelligence Help on Power Systems?

Power-system problem	Machine-learning task	Example output
Renewable or demand forecasting	Regression	Future wind, solar, or demand value
Fault or event detection	Classification	Normal or abnormal operating condition
Measurement analysis	Clustering or dimensionality reduction	Common patterns or unusual measurements
Operational control	Reinforcement learning or learning-assisted optimisation	Recommended control action

AI does not replace engineering knowledge

A statistically accurate prediction may still lead to an unsafe operational decision. Physical constraints, uncertainty, reliability requirements, and human oversight remain essential.

Outline

Power-System and Machine Learning Background

Course Overview

Introduction to Electric Power System

Machine Learning: Some Examples

Taxonomy of Machine Learning

Copyright and Reference

Copyright and Reference I

Some of the materials in this lecture note are taken from the following references:

- ▶ Murphy, K. P. Machine Learning—A probabilistic Perspective. The MIT Press, 2012. *A solid ML textbook with mathematical foundations.*
- ▶ Ian Goodfellow, Yoshua Bengio, and Aaron Courville, Deep Learning. The MIT Press, 2016². *One of the classics.*
- ▶ Bishop, Christopher M., and Nasser M. Nasrabadi. Pattern recognition and machine learning. Vol. 4. No. 4. New York: springer, 2006. *A little old and still useful.*

You are welcome (but not compulsory) to study online. For example,

- ▶ Machine learning specialization by Andrew Ng³.

²<https://www.deeplearningbook.org/>

³https://www.youtube.com/playlist?list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI

Outline

Supervised Learning

Overview

Linear Regression

Logistic Regression

Neural Networks

Support Vector Machine

Supervised Learning: Basic Idea I

Definition

In *supervised learning*, the goal is to learn a mapping from an input $\mathbf{x}$ to an output y , using a labelled dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N,$$

where:

- ▶ $\mathbf{x}_i$ is the input for sample i ;
- ▶ y_i is the corresponding label or target;
- ▶ N is the number of training samples.

The model learns a function

$$\hat{y} = f(\mathbf{x}; \theta),$$

where θ denotes the model parameters learned from data.

Inputs, Features, and the Design Matrix I

Each input $\mathbf{x}_i$ contains D features:

$$\mathbf{x}_i = \begin{bmatrix} x_i^{(1)} & x_i^{(2)} & \dots & x_i^{(D)} \end{bmatrix}^\top \in \mathbb{R}^D.$$

The inputs for all N samples are stored in the design matrix

$$X = \begin{bmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times D}.$$

In principal, x_i can contain complex information, such as an image, sentence, etc. In this lecture, we mainly focus on *time series* data.

Question

What features could be useful for forecasting solar generation?

Outline

Supervised Learning

Overview

Linear Regression

Logistic Regression

Neural Networks

Support Vector Machine

Linear Regression I

One of the most widely used models for regression is known as *linear regression*. This asserts that the response is a linear function of the input feature.

$$y(x) = w^T x + \epsilon = \sum_{j=1}^D w_j x_j + \epsilon$$

where D is the number of features and ϵ is the residual error between the linear predictions and the true response. Here, y is a scalar. w is the model parameter that you want to learn.

We often assume that ϵ has a Gaussian distribution. e.g., $\epsilon \sim \mathcal{N}(\mu, \sigma^2)$. Then the linear regression can be denoted as

$$p(y|x, \theta) = \mathcal{N}(y|\mu(x), \sigma^2(x))$$

Linear Regression II

In this case, it is assumed that y follows a conditional Gaussian distribution with mean given by

$$\mu(x) = w^T x$$

and the variance equals to the *fixed* noise $\sigma^2(x) = \sigma^2$.

For example, suppose the input is one-dimensional,

$$\mu(x) = w_0 + w_1 x = w^T x$$

where w_0 is the *bias* term, w_1 is the slope, and we can defined the vector $x = (1, x)$.

Questions

Can “linear” regression capture “nonlinear” relationships?

Linear Regression III

The linear regression can capture nonlinear relationship between x and y .

$$p(y|x, \theta) = \mathcal{N}(y|w^T \phi(x), \sigma^2)$$

where $\phi(x)$ is a nonlinear function of x . For example, a *polynomial regression* is defined as

$$\phi(x) = [1, x, x^2, \dots, x^d]$$

Linear Regression IV

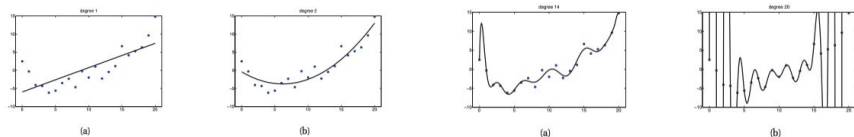


Figure: Linear regression (a) vs nonlinear regression (b).

Figure: Polynomial of degrees 14 (a) and 20 (b).

Questions

Does a perfect model (e.g. (b)) always give you good decision (compared to (a))?

Linear Regression V

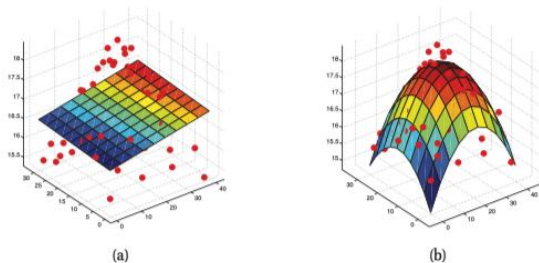


Figure: Linear regression on 2D data. (a). $\mu(x) = w_0 + w_1x_1 + w_2x_2$;
(b). $\mu(x) = w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2$

Learning Algorithm for Linear Regression I

The linear regression model is trained by minimising the distance between the forecast output and the target. The so-called *mean squared error* (MSE) loss is derived:

$$\min_w \text{MSE}(w) := \sum_{i=1}^N (y_i - w^T x_i)^2$$

which is also named as *least squares*.

The MSE loss can be denoted in matrix form:

$$\text{MSE}(w) = \frac{1}{2} \|y - Xw\|_2^2 = \frac{1}{2} (y - Xw)^T (y - Xw)$$

where $y \in \mathbb{R}^N$ is the forecasting target, $X \in \mathbb{R}^{N \times D}$ is the design matrix, and $w \in \mathbb{R}^D$ is the parameter.

Learning Algorithm for Linear Regression II

Taking the derivative with respect to w results in the first-order optimality condition:

$$X^T y - X^T X w = 0$$

Therefore, the least-square solution is given as:

$$\hat{w}_{ls} = (X^T X)^{-1} X^T y$$

Questions

Under what condition the $X^T X$ is invertible (non-singular)? What if it is not invertible?

Outline

Supervised Learning

Overview

Linear Regression

Logistic Regression

Neural Networks

Support Vector Machine

Logistic Regression I

Logistic regression is used for binary classification, even though its name contains the word “regression”.

Suppose the target is

$$y \in \{0, 1\},$$

where, for example,

$$y = \begin{cases} 1, & \text{abnormal operating condition, e.g., the grid is not stable,} \\ 0, & \text{normal operating condition, e.g., the grid is stable.} \end{cases}$$

Unlike linear regression, we do not directly predict an unrestricted numerical value. Instead, we predict the probability that the sample belongs to class 1.

First, logistic regression computes a linear score (recall the linear regression):

$$z = \mathbf{w}^T \mathbf{x}.$$

Logistic Regression II

This score can take any real value. We then transform it into a probability using the sigmoid function:

$$\mu(\mathbf{x}) = p(y = 1 \mid \mathbf{x}, \mathbf{w}) = \text{sigm}(\mathbf{w}^\top \mathbf{x}) \in (0, 1).$$

The sigmoid, or logistic function is defined as

$$\text{sigm}(\eta = \mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + \exp(-\eta)} = \frac{e^\eta}{1 + e^\eta}.$$

Interpretation

- ▶ If $\mathbf{w}^\top \mathbf{x}$ is large positive, the predicted probability is close to 1.
- ▶ If $\mathbf{w}^\top \mathbf{x}$ is large negative, the predicted probability is close to 0.
- ▶ If $\mathbf{w}^\top \mathbf{x} = 0$, the predicted probability is 0.5.

Logistic Regression III

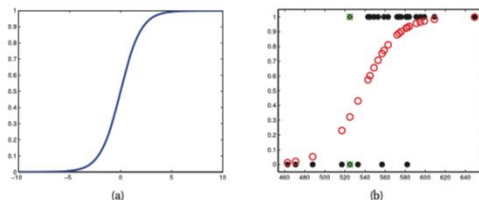


Figure: (a) The sigmoid function. (b) Logistic-regression probabilities and the corresponding binary classifications.

Question

Why sigmoid function can be used to describe the probability?

Logistic Regression IV

To convert the predicted probability into a class label, we choose a classification threshold. A common choice is 0.5:

$$\hat{y}_i = \begin{cases} 1, & p(y_i = 1 \mid x_i, \mathbf{w}) \geq 0.5, \\ 0, & p(y_i = 1 \mid x_i, \mathbf{w}) < 0.5. \end{cases}$$

Question

Is 0.5 always the best classification threshold?

Hint: For example, when detecting an unstable operating condition, would we prefer fewer missed detections or fewer false alarms?

Loss Function for Logistic Regression I

For each training sample, recall

$$\mu_i = p(y_i = 1 \mid \mathbf{x}_i, \mathbf{w}) = \text{sigm}(\mathbf{w}^\top \mathbf{x}_i).$$

Since $y_i \in \{0, 1\}$, the probability of observing y_i can be written compactly as

$$p(y_i \mid \mathbf{x}_i, \mathbf{w}) = \mu_i^{y_i} (1 - \mu_i)^{1-y_i}.$$

Assuming the training samples are independent, the likelihood of the entire training dataset is

$$p(\mathcal{D} \mid \mathbf{w}) = \prod_{i=1}^N \mu_i^{y_i} (1 - \mu_i)^{1-y_i}.$$

Loss Function for Logistic Regression II

Maximising the likelihood is equivalent to minimising the *binary cross-entropy loss*:

$$\text{CE}(\mathbf{w}) = - \sum_{i=1}^N [y_i \log \mu_i + (1 - y_i) \log(1 - \mu_i)].$$

Question

Can you observe how the CE-loss is derived?

How does cross-entropy work?

- ▶ If $y_i = 1$, the loss contains $-\log \mu_i$. The model is rewarded for making μ_i close to 1.
- ▶ If $y_i = 0$, the loss contains $-\log(1 - \mu_i)$. The model is rewarded for making μ_i close to 0.

Loss Function for Logistic Regression III

Question

Which prediction should receive the larger penalty?

$$y_i = 1, \quad \mu_i = 0.9$$

or

$$y_i = 1, \quad \mu_i = 0.1?$$

Question

Does CE have a closed-form solution, such as MSE loss in linear regression case? Why?

Learning Algorithm for Logistic Regression I

Unlike the ordinary least-squares problem in linear regression, logistic regression does not generally have a closed-form expression for the optimal parameter vector $\mathbf{w}$.

Therefore, we use a numerical optimisation algorithm. The gradient of the cross-entropy loss is

$$\nabla_{\mathbf{w}} \text{CE}(\mathbf{w}) = \sum_{i=1}^N (\mu_i - y_i) \mathbf{x}_i.$$

In matrix form,

$$\nabla_{\mathbf{w}} \text{CE}(\mathbf{w}) = X^{\top} (\boldsymbol{\mu} - \mathbf{y}),$$

where

$$\boldsymbol{\mu} = \text{sigm}(X\mathbf{w})$$

Learning Algorithm for Logistic Regression II

is applied element-wise.

Interpretation of the gradient

- ▶ Each training sample contributes $(\mu_i - y_i)\mathbf{x}_i$.
- ▶ If the predicted probability differs significantly from the true label, that sample produces a larger parameter update.

Learning Algorithm for Logistic Regression III

Numerical optimization techniques, such as *gradient descent* (first-order method) or *Newton's method* (second-order method) can be used. The *gradient descent* method is:

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \eta \nabla_{\mathbf{w}} \text{CE}(\mathbf{w}_k),$$

where:

- ▶ k is the iteration number;
- ▶ $\eta > 0$ is the learning rate;
- ▶ the gradient indicates the direction of steepest increase;
- ▶ therefore, we move in the negative-gradient direction.

Learning Algorithm for Logistic Regression IV

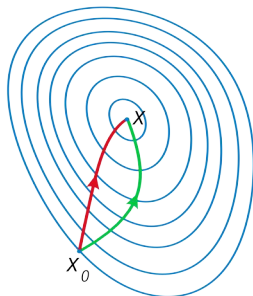


Figure: A comparison of gradient descent (green) and Newton's method (red) for minimizing a function (with small step sizes). Newton's method uses curvature information (i.e. the second derivative) to take a more direct route. Source: Wikipedia.

Learning Algorithm for Logistic Regression V

Question

What happens if the learning rate η is:

- ▶ too small?
- ▶ too large?

Key idea

Logistic regression combines three components:

linear score $\rightarrow$ sigmoid probability $\rightarrow$ binary classification.

Its parameters are learned by minimising the cross-entropy loss.

Outline

Supervised Learning

Overview

Linear Regression

Logistic Regression

Neural Networks

Support Vector Machine

From Linear Models to Neural Networks I

Linear and logistic regression start from a weighted sum:

$$z = \mathbf{w}^\top \mathbf{x} + b.$$

For regression, z may be used directly as the prediction. For binary classification, it is passed through a sigmoid function.

Basic idea

A neural network combines many simple computational units, called *neurons*, into layers. Each neuron computes

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad h = \sigma(z),$$

where $\sigma(\cdot)$ is a nonlinear activation function.

Common activation functions include

$$\text{sigm}(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \text{ReLU}(z) = \max(0, z).$$

From Linear Models to Neural Networks II

Why is the activation function important?

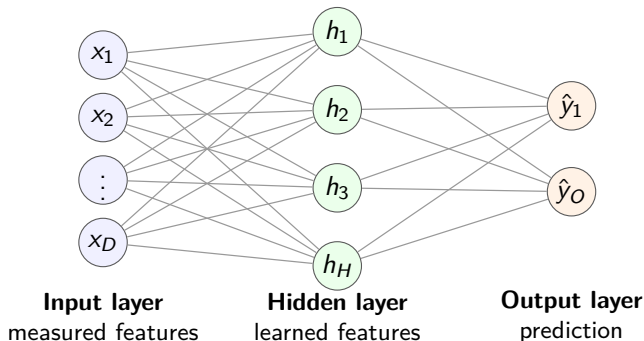
Without nonlinear activation functions, stacking several layers would still be equivalent to a single linear transformation.

You will also find out many NN settings are the same as the linear regression and logistic regression!

Feedforward Neural Network I

A feedforward neural network contains an input layer, one or more hidden layers, and an output layer. For one hidden layer,

$$\mathbf{h} = \sigma(W_1\mathbf{x} + \mathbf{b}_1), \quad \hat{\mathbf{y}} = g(W_2\mathbf{h} + \mathbf{b}_2).$$



Feedforward Neural Network II

Key idea

The hidden layer learns intermediate features $\mathbf{h}$ from the data instead of requiring every useful feature to be designed manually.

Neural Networks for Different Learning Tasks I

The output layer and loss function depend on the required task.

Task	Output layer	Typical loss
Regression	$\hat{\mathbf{y}} = W\mathbf{h} + \mathbf{b}$	Mean squared error
Binary classification	$\hat{y} = \text{sigm}(z)$	Binary cross-entropy
Multiclass classification	$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$	Cross-entropy

For K classes,

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K.$$

Neural Networks for Different Learning Tasks II

Power-system examples

- ▶ Wind-power forecasting: regression.
- ▶ Stable or unstable operating point: binary classification.
- ▶ Normal operation, line fault, or generator fault: multiclass classification.

Question

Which output activation and loss function should be used for each of the three examples above?

How Does a Neural Network Learn? I

Let all trainable parameters be collected in θ . Training solves

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f(\mathbf{x}_i; \theta), y_i).$$

The training process repeatedly performs:

1. **Forward pass:** calculate $\hat{y}_i = f(\mathbf{x}_i; \theta)$.
2. **Loss evaluation:** compare $\hat{y}_i$ with the true target y_i .
3. **Backpropagation:** use the chain rule to compute $\nabla_{\theta} \mathcal{L}$.
4. **Parameter update:**

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} \mathcal{L}(\theta_k).$$

How Does a Neural Network Learn? II

Backpropagation

Backpropagation efficiently determines how each weight and bias contributes to the prediction error by propagating gradient information backwards through the network.

Question

What happens if the learning rate η is too small or too large?

Power-System Example and Engineering Considerations

Consider one-hour-ahead solar-generation forecasting:

$$\mathbf{x}_t = [P_t^{\text{solar}}, P_{t-1}^{\text{solar}}, I_{t+1}, T_{t+1}, C_{t+1}, \text{time}_{t+1}]^\top, \quad y_t = P_{t+1}^{\text{solar}},$$

where I , T , and C denote irradiance, temperature, and cloud cover.

A simple neural-network forecaster is

$$\mathbf{h}_t = \text{ReLU}(W_1 \mathbf{x}_t + \mathbf{b}_1), \quad \hat{P}_{t+1}^{\text{solar}} = W_2 \mathbf{h}_t + b_2.$$

Advantages

- ▶ captures nonlinear relationships;
- ▶ learns useful features automatically;
- ▶ supports multiple inputs and outputs;
- ▶ is fast to evaluate after training.

Limitations

- ▶ may require large datasets;
- ▶ can overfit (what does this mean?);
- ▶ may be difficult to interpret;
- ▶ can fail under distribution shift (what does this mean?).

Outline

Supervised Learning

Overview

Linear Regression

Logistic Regression

Neural Networks

Support Vector Machine

Support Vector Machine: Overview I

A *support vector machine* (SVM) is a supervised-learning method used mainly for classification. Its central idea is to find a decision boundary that separates the classes with the *largest possible margin*. For binary classification, let $y_i \in \{-1, +1\}$. A linear SVM predicts

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} - b),$$

where $\mathbf{w}^\top \mathbf{x} - b = 0$ defines the separating hyperplane.

Key idea

Among all separating boundaries, SVM chooses the one that stays as far as possible from the nearest samples in both classes.

Support Vector Machine: Overview II

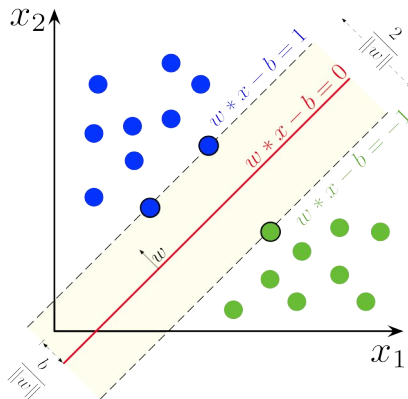


Figure: SVM is a max-margin classification. The nearest training samples are the *support vectors*.

Support Vector Machine: Mathematical Formulation I

Assume first that the training data are perfectly linearly separable.
Because the scale of $\mathbf{w}$ and b is arbitrary, the two margin boundaries can be written as

$$\mathbf{w}^\top \mathbf{x} - b = 1, \quad \mathbf{w}^\top \mathbf{x} - b = -1.$$

Their distance is $2/\|\mathbf{w}\|_2$. Hence, maximising the margin is equivalent to minimising $\|\mathbf{w}\|_2^2$.

We also need to put data points outside of the margin area, e.g.,

$$\begin{aligned} w^\top x_i - b &\geq 1, & y_i &= 1 \\ w^\top x_i - b &\leq -1, & y_i &= -1 \end{aligned}$$

Correct classification outside the margin requires

$$y_i(\mathbf{w}^\top \mathbf{x}_i - b) \geq 1, \quad i = 1, \dots, N.$$

Support Vector Machine: Mathematical Formulation II

Therefore, the hard-margin SVM becomes the following *optimization* problem:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i - b) \geq 1, \quad i = 1, \dots, N. \end{aligned}$$

Interpretation

The objective increases the margin, while the constraints require every training sample to be correctly classified and outside the margin.

Support Vector Machine: Mathematical Formulation III

SVM Variants

- ▶ The hard-margin assumption is often unrealistic because real datasets may contain noise, outliers, or overlapping classes. In this case, we can use soft-margin SVM to allow misclassification.
- ▶ A linear decision boundary may not be sufficient for more complex data. In this case, we can use kernel SVM to capture the nonlinear transformed features.
- ▶ In the coding tutorial (if we have time), different settings are compared using a Python machine learning package called `sklearn` including different kernel types, different soft-margin parameters, etc.

Outline

Unsupervised Learning

Overview

Principal Component Analysis

k-means Clustering

Unsupervised Learning: Definition I

Questions

Given a set of power system sensor measurement data, let us consider a task to separate samples with irregular voltage profile. What can you do if there is no label given?

Definition

In *unsupervised learning*, there is only given inputs $\mathcal{D} = \{x_i\}_{i=1}^N$ and the goal is to find interesting patterns in the data. This is a much less well-defined problem, since we are not told what kinds of pattern to look for.

Unsupervised Learning: Definition II

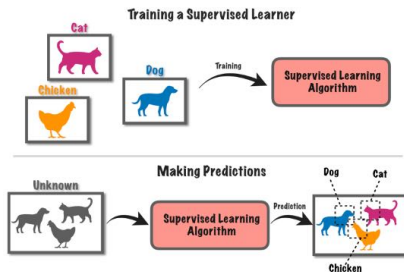


Figure: Supervised vs unsupervised learning.

Some examples include *principal component analysis (PCA)*, *k-means*, etc.

Unsupervised Learning: Definition III

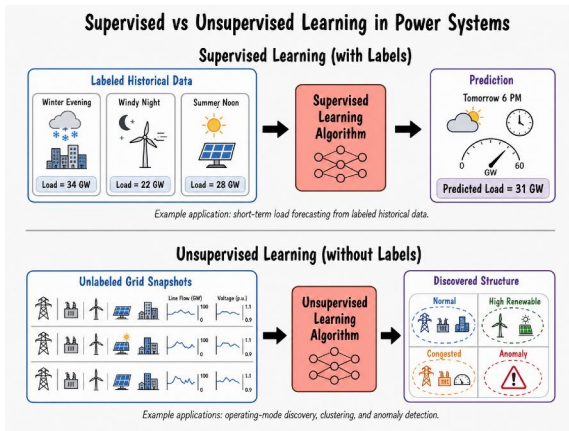


Figure: Supervised learning vs unsupervised learning in power systems.

Outline

Unsupervised Learning

Overview

Principal Component Analysis

k-means Clustering

PCA I

Imagine you have a dataset with 10,000 number of features. And you would like to “compress” the dataset into lower dimensions.

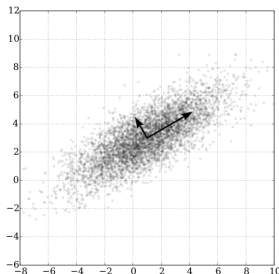
Question

When is a dimension reduction useful?

Question

Consider the 2D data shown below. For the two-marked directions, which direction contains more information, why?

PCA II



PCA⁴ is a *linear dimensionality reduction* method that can project the data into fewer dimensions (the D) in a way that most of the information of the data is kept!

- Summary: Minimize the dimension while keeping the information.

PCA III

Consider that you have some sensor measurement data that has a large number of features, e.g., D is very large,

- ▶ You would like to visualize the data. Then...
- ▶ You would like to first extract useful information, then apply ML algorithms. Then...

Hint: It is always a great practice to visualize your data (in low dimensions) to see if it has some common mode.

Question

Explain why PCA is an unsupervised learning.

Question

Does PCA project onto the existing features?

⁴<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

Outline

Unsupervised Learning

Overview

Principal Component Analysis

k-means Clustering

k-means Clustering I

As the name suggests, the k-means⁵ method divides the training set into k different classes in an *unsupervised* way (without label).

The k-means algorithm works as follows

1. Determine the number of clusters k .
2. Initialize k (random) different center of clusters $\{c_1, \dots, c_k\}$.
3. Assign each training example to cluster i , where i is the index of the *nearest* center c_i .
4. Each center c_i is *updated to the mean* of all training examples assigned to cluster i (centroid).
5. Repeat step (3)-(4).

Question

For the task of detecting normal and abnormal sensor measurements from unlabeled data, how many clusters (k) should be assumed in step (1)?

k-means Clustering II

k-means Clustering

Example: grouping similar operating conditions or load profiles

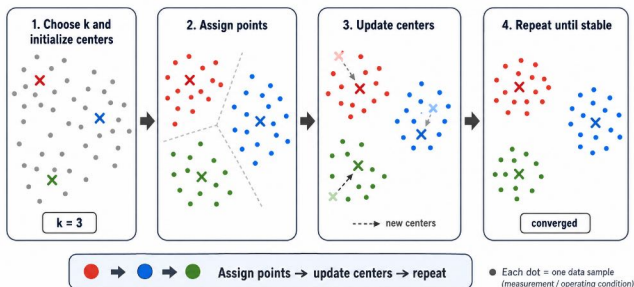


Figure: Visualization of k-means algorithm.

⁵https:

//scikit-learn.org/1.5/modules/generated/sklearn.cluster.KMeans.html

Outline

Reinforcement Learning Overview Only

Reinforcement Learning I

Unlike supervised learning, reinforcement learning does not require the correct action to be provided for every situation. Instead, an *agent* learns how to act by repeatedly interacting with an *environment* and receiving rewards or penalties.

state $\mathbf{s}_t \longrightarrow$ action $\mathbf{a}_t \longrightarrow$ environment $\longrightarrow$ reward r_t and next state $\mathbf{s}_{t+1}$.

Battery charging problem

The agent learns when to charge or discharge the battery so that the total cost over many time steps is reduced while respecting operational limits.

Reinforcement Learning II

Main components

- ▶ **State s_t :** information describing the current situation;
- ▶ **Action a_t :** the decision made by the agent;
- ▶ **Reward r_t :** feedback indicating whether the action was useful;
- ▶ **Policy:** a rule mapping states to actions.

Example: battery energy management

At each hour, the controller observes:

$$s_t = [\text{demand}, \text{renewable}, \text{price}].$$

It then chooses an action:

$$a_t \in \{\text{charge}, \text{discharge}, \text{idle}\}.$$

A possible reward is:

$$r_t = -\text{electricity cost} - \text{constraint penalties}.$$

Reinforcement Learning III

Question

Why might charging the battery during one hour produce a negative immediate reward, but still lead to a better total reward later?

Agent–Environment Interaction

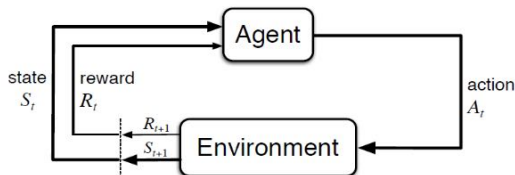


Figure: The agent learns by repeatedly interacting with the environment.

Battery example

Agent: battery controller,

Environment: battery, demand, renewables, and electricity price,

Action: charge, discharge, or remain idle,

Reward: low operating cost with safe battery operation.

Outline

Performance Measure

Performance Measure

Underfit and Overfit

Regularization

Hyperparameters

Validation Set

Summary

Performance Measure I

In order to evaluate the abilities of a machine learning algorithm, we must design a quantitative but straightforward measure of its performance.

For *regression task*, we can use the mean squared error (MSE) as defined before.

For the binary classification task on sensor measurement, we can define more performance measures, including

- ▶ *True positive rate (TPR)*: the proportion of abnormal measurements (class 1) that are correctly marked (as class 1). You can also call it the *detection rate*.
- ▶ *False positive rate (FPR)*: the proportion of normal measurement (class 0) that are incorrectly marked (as class 1). You can also call it *false alarm*.

Performance Measure II

- ▶ *True negative rate (TNR)*: the proportion of normal measurements (class 0) that are correctly marked (as class 0).
- ▶ *False negative rate (FNR)*: the proportion of abnormal measurements (class 1) that are incorrectly marked as (class 0). You can also call it *missing alarm rate*.

How to remember? FPR: falsely classified as positive, etc.

To systematically include all the measures, the F_1 score is defined as

$$F_1 = \frac{2TP}{2TP + FP + FN}$$

Performance Measure III

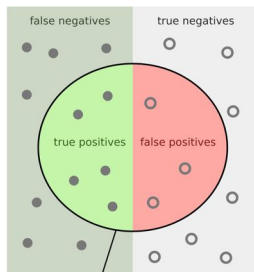


Figure: Set-based illustration of binary-classification outcomes. The left region contains actual positive samples, corresponding to abnormal measurements, while the right region contains actual negative samples, corresponding to normal measurements. The circle contains samples predicted as positive. The overlap and non-overlap regions represent TP, FP, TN, and FN.

Performance Measure IV

Application	False Positive	False Negative
Fault detection	A healthy component is incorrectly declared faulty, causing unnecessary inspection or isolation.	A real fault is not detected, potentially delaying protection or corrective action.
Transient-stability prediction	A stable operating point is classified as unstable, leading to unnecessarily conservative operation.	An unstable operating point is classified as stable, potentially causing loss of synchronism or cascading outages.
Voltage-security assessment	A secure state is classified as insecure, causing unnecessary re-dispatch or load curtailment.	An insecure state is classified as secure, leaving the system vulnerable to voltage collapse.
Cyberattack detection	Normal measurements are identified as malicious, producing excessive alarms.	Manipulated measurements are accepted as normal and may corrupt state estimation or operational decisions.
Line-overload prediction	A non-congested line is predicted to overload, causing unnecessary preventive redispatch.	A future overload is missed, increasing the risk of protection activation or equipment damage.

Performance Measure V

Question

Can you identify the TPR, FPR, TNR, and FNR in the above plot?

Question: Fault Detection

A fault detector correctly detects 92 out of 100 faults, but it also raises alarms for 45 out of 900 normal samples. What are its TPR and FPR?

Question

For power system stability prediction, which error is generally more dangerous: classifying a stable case as unstable, or classifying an unstable case as stable? Why?

Outline

Performance Measure

Performance Measure

Underfit and Overfit

Regularization

Hyperparameters

Validation Set

Summary

Overfitting and Underfitting I

Having good accuracy on the training dataset is not the ultimate goal of machine learning. The central challenge in machine learning is that we must perform well on *new, unseen* inputs.

The ability to perform well on previously unobserved inputs (you can call it test set) is called *generalization*.

When training a machine learning model, we *only* have access to a training set, we can compute some error measure on the training set called the *training error*, and we reduce this training error as described in the linear regression case through optimization.

$$\text{MSE}^{\text{train}} = \|X^{\text{train}}w - y^{\text{train}}\|_2^2$$

Overfitting and Underfitting II

What separates machine learning from optimization is that we want the generalization error, also called the *test error*, to be low as well.

$$\text{MSE}^{\text{test}} = \|X^{\text{test}}w - y^{\text{test}}\|_2^2$$

Overfitting and Underfitting III

Therefore, the factors determining how well a machine learning algorithm will perform are its ability to:

1. Make the training error small.
2. *Make the gap between training and test error small.*

Definition

Underfit occurs when the model is not able to obtain a sufficiently low error value on the training set. *Overfit* occurs when the gap between the training error and test error is too large.

Overfitting and Underfitting IV

We can control whether a model is more likely to overfit or underfit by altering its *capacity*.

- ▶ Models with insufficient capacity are unable to solve complex tasks.
- ▶ Models with high capacity can solve complex tasks, but when their *capacity is higher than needed* to solve the present task, they may overfit.

One way to control the capacity of a learning algorithm is by choosing its *hypothesis space*, the set of functions that the learning algorithm is allowed to select as being the solution.

As mentioned before, the linear regression algorithm has the set of all linear functions of its input as its hypothesis space. We can generalize linear regression to include polynomials (polynomial regression), rather than just linear functions, in its hypothesis space. Doing so increases the capacity of the model.

Overfitting and Underfitting V

Normally, we can start to choose model with lower capacity (complexity).
For example, a linear regression with polynomial of degree one:

$$\hat{y} = b + wx$$

By introducing x^2 as another feature, we increase the complexity of the regression model:

$$\hat{y} = b + w_1x + w_2x^2$$

We can continue to add more powers of x , for example a polynomial of degree of 9:

$$\hat{y} = b + \sum_{i=1}^9 w_i x^i$$

Overfitting and Underfitting VI

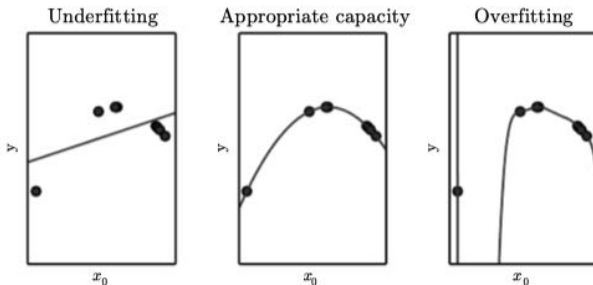


Figure: Underfit vs overfit

Overfitting and Underfitting VII

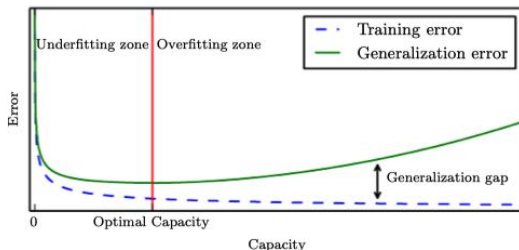


Figure: Typical relationship between capacity and error.

Question

We still have many unsolved problems. For example, we don't have access to the test set... Then how can we know the generalization error (or the performance on test dataset) during training?

Outline

Performance Measure

Performance Measure

Underfit and Overfit

Regularization

Hyperparameters

Validation Set

Summary

Regularization I

Degree of complexity: So far, the only method of modifying a learning algorithm is to increase or decrease the model's representational capacity by adding or removing the degree of the features.

Larger hypothesis space: Another way to control the performance (capacity) of the algorithms is to choose what kind of functions that can better fit on the data. For example, linear regression would not perform very well if we try to use it to fit on $\sin(x)$ from x .

Regularization: We can introduce a *preference* in the learning algorithm for certain solutions within its hypothesis space. In other words, the algorithm will choose a less preferred (*more complex*) solution only if it fits the training data *significantly better* than the preferred (simpler) one.

Regularization II

We can include the *weight decay* on the linear regression loss for *regularization* purpose.

$$\text{MSE}^{\text{reg}} = \text{MSE}^{\text{train}} + \lambda w^T w$$

where λ is a value chosen before the final training (a hyperparameter).

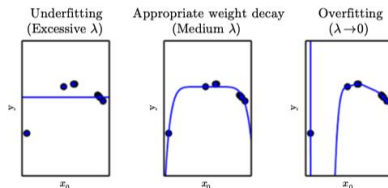


Figure: Different λ on the overfitted polynomial.

Questions

What actually happens if $\lambda \rightarrow 0$ and ∞ ?

Outline

Performance Measure

Performance Measure

Underfit and Overfit

Regularization

Hyperparameters

Validation Set

Summary

Hyperparameters I

Definition

Most machine learning algorithms have several settings that we can use to control the behavior of the learning algorithm. These settings are called *hyperparameters*.

NOTE: The values of hyperparameters are *not* adapted by the learning algorithm itself. Why?

The hyperparameters significantly influence the performance of the ML model. For example,

1. In the polynomial regression, $\phi(x) = [1, x, x^2, \dots, x^d]$, the degree of the polynomial, which represents the *capacity* hyperparameter.
2. In the regression with weight decay, $\text{MSE}^{\text{reg}} = \text{MSE}^{\text{train}} + \lambda w^T w$, the λ value used to control the *strength* of weight decay is another example of hyperparameter.
3. Whenever you train an ML model, there always exists a set of hyperparameters which should carefully play with.

Hyperparameters II

SVC

```
class sklearn.svm.SVC(*, C=1.0, kernel='rbf', degree=3, gamma='scale',  
coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=200,  
class_weight=None, verbose=False, max_iter=-1, decision_function_shape='ovr',  
break_ties=False, random_state=None)
```

[\[source\]](#)

Figure: Sklearn SVM function. There are many hyperparameters to be set. Some of them can be more impactful than the other. You will have example on this.

Question

In the simplest setting, it is not appropriate to learn the hyperparameters on the training set. Based on our discussion on overfitting, explain why.

Outline

Performance Measure

Performance Measure

Underfit and Overfit

Regularization

Hyperparameters

Validation Set

Summary

Validation Set I

Recall a previous problem,

Question

We still have many unsolved problems. For example, we don't have access to the test set... Then how can we know the generalization error (or the performance on test dataset) during training?

We would like to *estimate* the generalizability of the ML model in *an unknown dataset*. The solution is to separate a held-out set from the *training dataset*, called *validation set*. Important notice:

1. The validation set should NOT be used to train the model!
2. NO test data should be included in the validation set!

Specifically, we split the training data into two *disjoint* subsets. One of these subsets is used to learn the parameters (e.g., 80%). The other subset is our validation set (e.g., 20%), used to estimate the generalization error during or after training, allowing for the hyperparameters to be updated accordingly.

Validation Set II

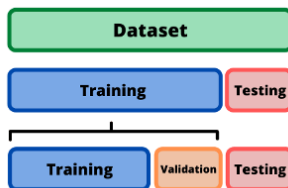


Figure: Train, validation, and test dataset.

Given labeled dataset. The procedure could be:

1. Split the training dataset into 'training dataset' and validation set with a predefined ratio (e.g., 0.8 vs 0.2).
2. Determine an ML model you would like to train on.
3. Set a list of value of all hyperparameters. E.g., $\lambda = [0, 0.01, 0.001]$, $d = 1, 2, \dots$ etc.

Validation Set III

4. Train the ML model under each of the hyperparameters combination, e.g. $(\lambda = 0, d = 1)$, $(\lambda = 0.01, d = 2)$, etc on the *training dataset*. This processing is also called *grid search*.
5. Evaluate the performance of all trained models with different hyperparameter combinations on *validation set*. The one with the best performance is the final model.

Basically, you never touch (you do not have actually) on the test set during model training.

Validation Set IV

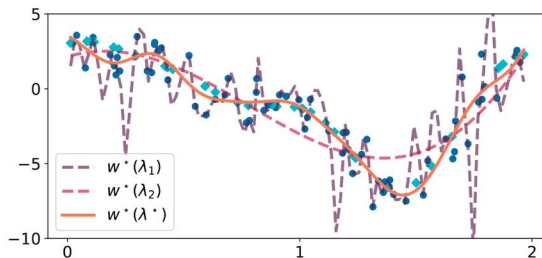


Figure: Tuning hyperparameter λ based on the performance on the **validation set**. This example is taken from [Blondel and Roulet, 2024]. Blue dots are training samples and cyan dots are validation samples.

Question

Can you tell which fitting curve is over-fitted and under-fitted?

Outline

Performance Measure

Performance Measure

Underfit and Overfit

Regularization

Hyperparameters

Validation Set

Summary

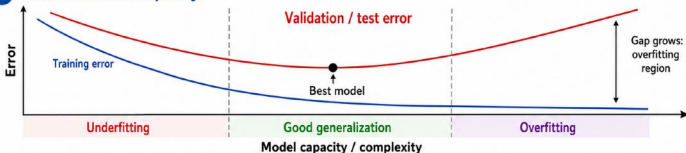
Summary

Understanding Underfitting, Overfitting, and Generalization

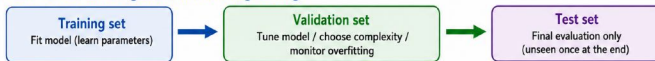
1 Model fit examples



2 Error vs model capacity



3 How do we estimate generalization during training?



We do not use the test set during training. We use a validation set (or cross-validation) to estimate generalization.



Low training error alone is not enough; a good model also keeps the validation/test error low on unseen data.

Outline

Special Topic One: Time Series Foundation Model

No Free Lunch Theorem

What is Foundation Model?

Time Series Foundation Model

Amazon Chronos

No Free Lunch Theorem I

- ▶ “All models are wrong, but some models are useful.”
- ▶ There is no single best model that works optimally for all kinds of problems. The reason is that a set of assumptions that works well in one domain may *generalize* poorly in another.
- ▶ As a consequence of the *no free lunch theorem*, we need to develop many different types of models, to cover the wide variety of data that occurs in the real world. And for each model, there may be many different algorithms we can use to train the model.
- ▶ But with the development of foundation model, the No Free Lunch Theorem “seems” to be less accurate.

Outline

Special Topic One: Time Series Foundation Model

No Free Lunch Theorem

What is Foundation Model?

Time Series Foundation Model

Amazon Chronos

Foundation Model I

All the techniques we discussed earlier require training by yourself. This is because one model is typically only good at one task.

Definition

A *foundation model* is a *large-scale, pre-trained* model (based on deep learning) that learns general-purpose representations from *massive, diverse datasets* and can be *adapted* (fine-tuned or prompted) to perform many downstream tasks.

- ▶ Trained on broad and heterogeneous data (e.g., text, image, code, multimodal data, etc);
- ▶ Use self-supervised or unsupervised learning (e.g. predicting masked tokens, next word, next frame, etc.): predict on itself.
- ▶ Transferable to many domains with prompting (zero-shot) or fine-tuning (few-shot).

Foundation Model II

- ▶ **LARGE:** Billions of parameters.

Questions

Can GPT-5 or Deepseek be seen as a foundation model?

Suppose you have a model like GPT-5. You can immediately adapt it for:

- ▶ Writing technical reports (text generation).
- ▶ Extracting entities from energy market documents (information extraction).
- ▶ Explaining optimization models (reasoning).

You don't train it from scratch: you just prompt or fine-tune it → this reusability makes it a foundation model.

Outline

Special Topic One: Time Series Foundation Model

No Free Lunch Theorem

What is Foundation Model?

Time Series Foundation Model

Amazon Chronos

Time Series Foundation Model (TSFM) I

Definition

A time series foundation model is a large pre-trained model designed to understand and predict *temporal* patterns across diverse time-dependent datasets. It is trained on massive collections of time series from many domains (e.g., finance, energy, weather, health) and can generalize to new, unseen time series without retraining from scratch.

Questions

Does TSFM follow the properties of general FM?

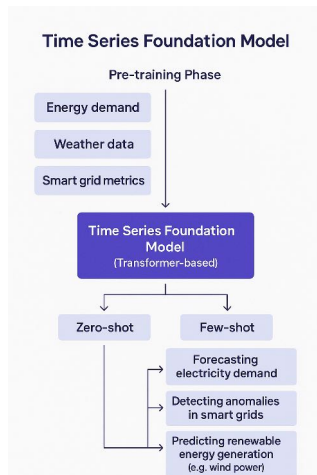


Figure: The concept of TSFM (image generated by GPT-5).

Outline

Special Topic One: Time Series Foundation Model

No Free Lunch Theorem

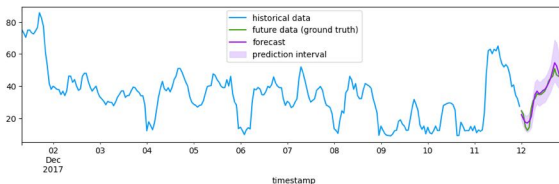
What is Foundation Model?

Time Series Foundation Model

Amazon Chronos

Amazon Chronos I

- ▶ Chronos is a family of pretrained time series forecasting models open sourced by Amazon. The latest version supports zero-shot univariate, multivariate, and covariate-informed forecasting tasks (Can you guess what is covariate?). The largest one has 710M parameters.
- ▶ GitHub repository:
<https://github.com/amazon-science/chronos-forecasting>.
- ▶ A zero-shot example for energy price forecasting.



Outline

Special Topic Two: DATA? DATA!

Why Data is Important?

From Raw Data to an ML Dataset

European's Effort on Data Transparency

Why Data is Important? I

- ▶ All the previous discussions assume that you already have *sufficient* dataset with *good quality*.
- ▶ For energy systems, some of the most important data include power consumption and generation, energy price (day-ahead and real-time) in an open market, and operational status such as transmission line congestion level.
- ▶ Believe it or not, high quality data is not easy to get. And there is a famous one: “Garbage in, garbage out” (GIGO).
- ▶ But luckily, the EU (including UK) has enforced many regulations for opening data.

Outline

Special Topic Two: DATA? DATA!

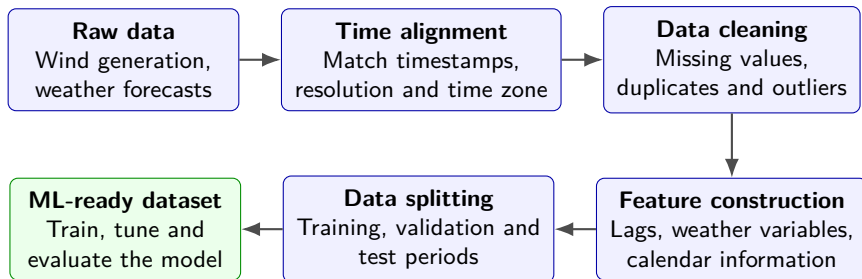
Why Data is Important?

From Raw Data to an ML Dataset

European's Effort on Data Transparency

From Raw Power-System Data to an ML Dataset

Example task: one-hour-ahead wind-power forecasting.



Key message

Downloading data is only the first step. The raw records must be converted into samples containing information that is available at the time when the prediction is made.

Outline

Special Topic Two: DATA? DATA!

Why Data is Important?

From Raw Data to an ML Dataset

European's Effort on Data Transparency

European's Effort on Data Transparency I

- ▶ Open energy data is essential for ensuring transparency, market efficiency, and innovation in the energy sector.
- ▶ It allows regulators, researchers, and consumers to better understand how electricity is produced, transmitted, and consumed, fostering trust and enabling smarter system planning and policy design.
- ▶ The European Union has established several key regulations to enforce this transparency, most notably Regulation (EU) No 543/2013, which mandates that electricity market participants, such as transmission system operators (TSOs), power plant operators, and large consumers, publish detailed operational data on generation, demand, outages, and balancing.

European's Effort on Data Transparency II

- ▶ Complementary frameworks like REMIT (Regulation 1227/2011) ensure integrity in wholesale energy markets, while newer laws such as the Energy Efficiency Directive (2023/1791) and the EU Data Act (2023/2854) extend transparency and data-sharing obligations to other parts of the energy ecosystem.
- ▶ Most of this data is publicly available and freely accessible through the *ENTSO-E Transparency Platform*, which serves as the central hub for Europe-wide electricity system operational data.

Where to Find the Power System Data I

- ▶ Electricity generation, transportation and consumption for the European market: <https://newtransparency.entsoe.eu/> is the main source for obtain market, load, generation, transmission, and balancing data.
- ▶ Weather dataset is important to forecast nearly all factors in electricity market. You may find it useful in ERA5 dataset <https://cds.climate.copernicus.eu/datasets>.
- ▶ More detailed operation and market data may be found on regional operators website, such as FinGrid and Elia.

Reference I



Blondel, M. and Roulet, V. (2024).

The elements of differentiable programming.

arXiv preprint arXiv:2403.14606.



Hamann, H. F., Brunschweiler, T., Gjorgiev, B., Martins, L. S., Puech, A., Varbella, A., Weiss, J., Bernabe-Moreno, J., Massé, A. B., Choi, S., et al. (2024).

A perspective on foundation models for the electric power grid.

arXiv preprint arXiv:2407.09434.

Outline

Appendix

Linear Regression: Maximum Likelihood Estimation

Soft-margin SVM and Kernel SVM

Linear Regression: Maximum Likelihood Estimation I

Linear regression can be modelled and trained by *maximum likelihood estimation* (MLE).

$$\hat{\theta} = \arg \max_{\theta} \log p(\mathcal{D}|\theta)$$

Assuming that the training examples are independent and identically distributed (iid), the log-likelihood is represented as

$$\log p(\mathcal{D}|\theta) = \log \prod_{i=1}^N p(y_i|x_i, \theta) = \sum_{i=1}^N \log p(y_i|x_i, \theta)$$

Recall that a Gaussian distribution is assumed:

$$p(y|x, \theta) = (2\pi\sigma^2)^{-\frac{1}{2}} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Linear Regression: Maximum Likelihood Estimation II

Plugging into the log likelihood:

$$\ell(\theta) = -\frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - w^T x_i)^2 - \cancel{\frac{N}{2} \log 2\pi\sigma^2} \xrightarrow{\text{cancel}}$$

In the case when σ is fixed, the second terms is removed.

Question

Based on the maximum likelihood estimation, can you derive the MSE loss by yourself?

Outline

Appendix

Linear Regression: Maximum Likelihood Estimation

Soft-margin SVM and Kernel SVM

Soft-margin SVM and Kernel SVM I

The hard-margin assumption is often unrealistic because real datasets may contain noise, outliers, or overlapping classes.

Soft-margin SVM

Soft-margin SVM allows some samples to enter the margin, or even to be misclassified, while penalising these violations.

Introducing slack variables $\xi_i \geq 0$, the soft-margin formulation is

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i - b) \geq 1 - \xi_i, \quad i = 1, \dots, N, \\ & \xi_i \geq 0, \quad i = 1, \dots, N. \end{aligned}$$

Soft-margin SVM and Kernel SVM II

Here, $C > 0$ controls the trade-off between a wide margin and fewer training violations:

C large $\Rightarrow$ strong penalty on classification errors,

C small $\Rightarrow$ more violations allowed for a wider margin.

Question

What might happen when C is extremely large? What might happen when C is extremely small?

Soft-margin SVM and Kernel SVM III

A linear decision boundary may not be sufficient for more complex data.

Kernel SVM

A kernel allows an SVM to construct a nonlinear decision boundary without explicitly computing all nonlinear transformed features.

Common choices include the linear, polynomial, and radial basis function (RBF) kernels.

Practical point

SVMs are sensitive to feature scales because their decisions depend on distances and margins. Input features should normally be standardised before training.

Take-home message

SVM chooses a separating boundary with a large margin. Soft margins handle imperfect data, while kernels allow nonlinear classification.